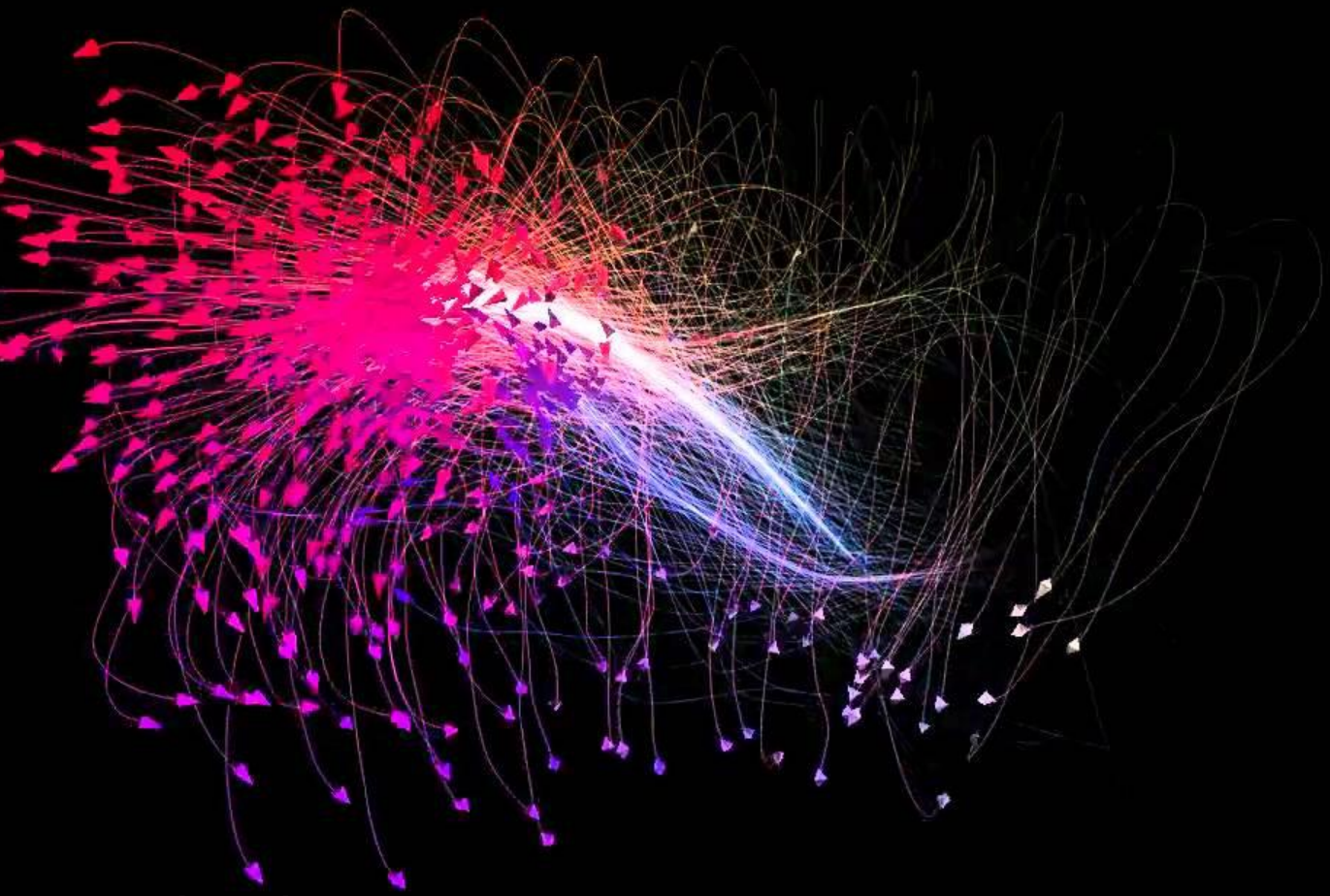
Computational Intelligence

Hard Problems



Anikó Ekárt & Alina Patelli
(Based on material by Peter Lewis)

Aston University

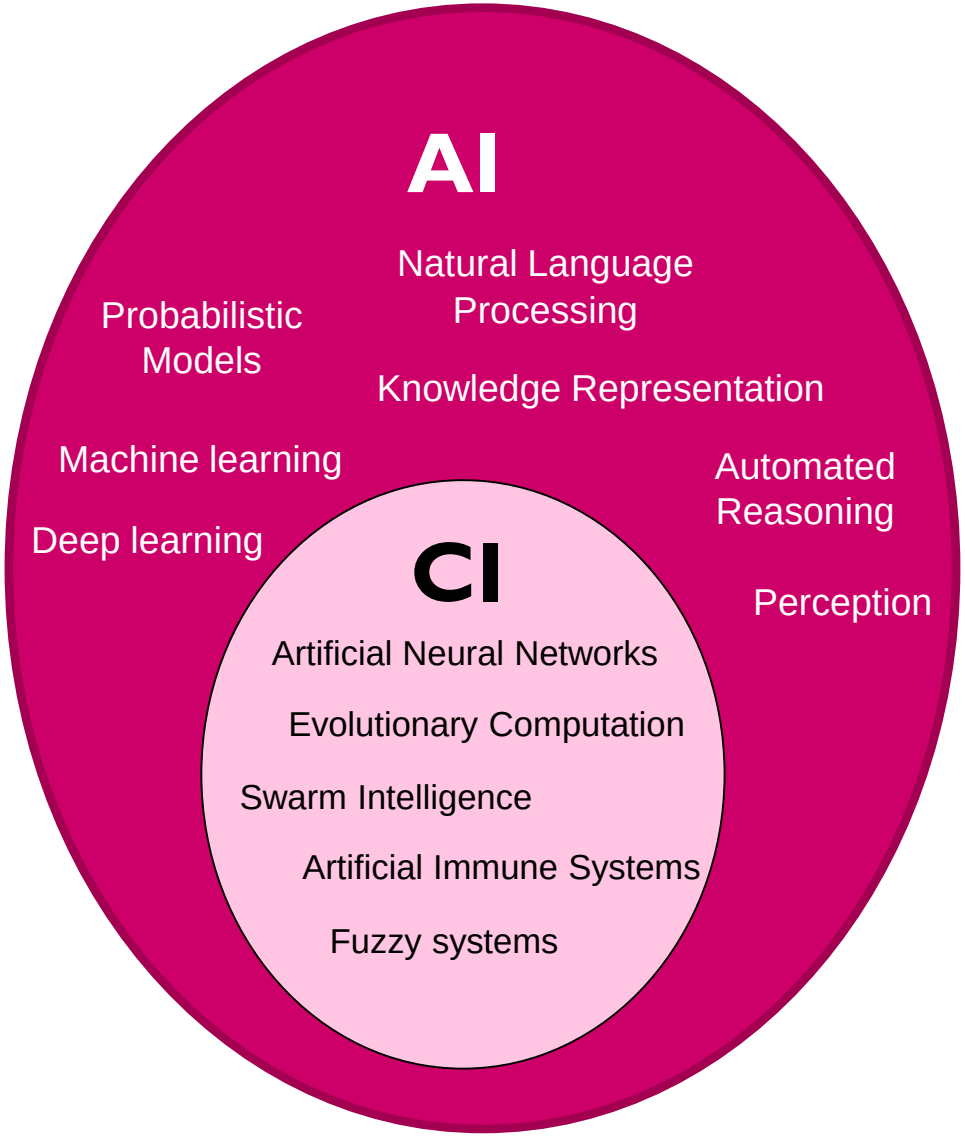
Overview of this lecture

- By the end of this lecture you should:
  - Understand what makes a “hard” problem, and be able to discuss the relative hardness of problems.
  - Know about the two main problems that will be used in this module:
    - The Travelling Salesman Problem, and
    - The Antenna Array Design Problem.
  - Appreciate the scale and hardness of each of these two problems, from the point of view of implementing solutions.

Quick recap

Computational Intelligence is...

- A subset of AI.
- That is mainly concerned with “bottom-up”, nature-inspired techniques.
- To enable computers to solve *hard** problems.



“Hard” has a specific technical meaning (more on that later), but is informally often thought of as something that only intelligent beings can do (and possibly find hard to do or to explain), or something which a computer program cannot do in a reasonable amount of time.

So what makes a problem hard?



Finding a needle in a haystack seems hard:

- Need to check each piece of hay **one-by-one**: “is this a needle?”

- Key feature:

easy to check if a piece **is** a needle, but hard to **find** the needle.

So what makes a problem hard?

Crack a combination lock…



- Need to check each combination **one-by-one**: “does it open?”

- Key feature:

easy to check if a combination is correct, but hard to **find** the combination.

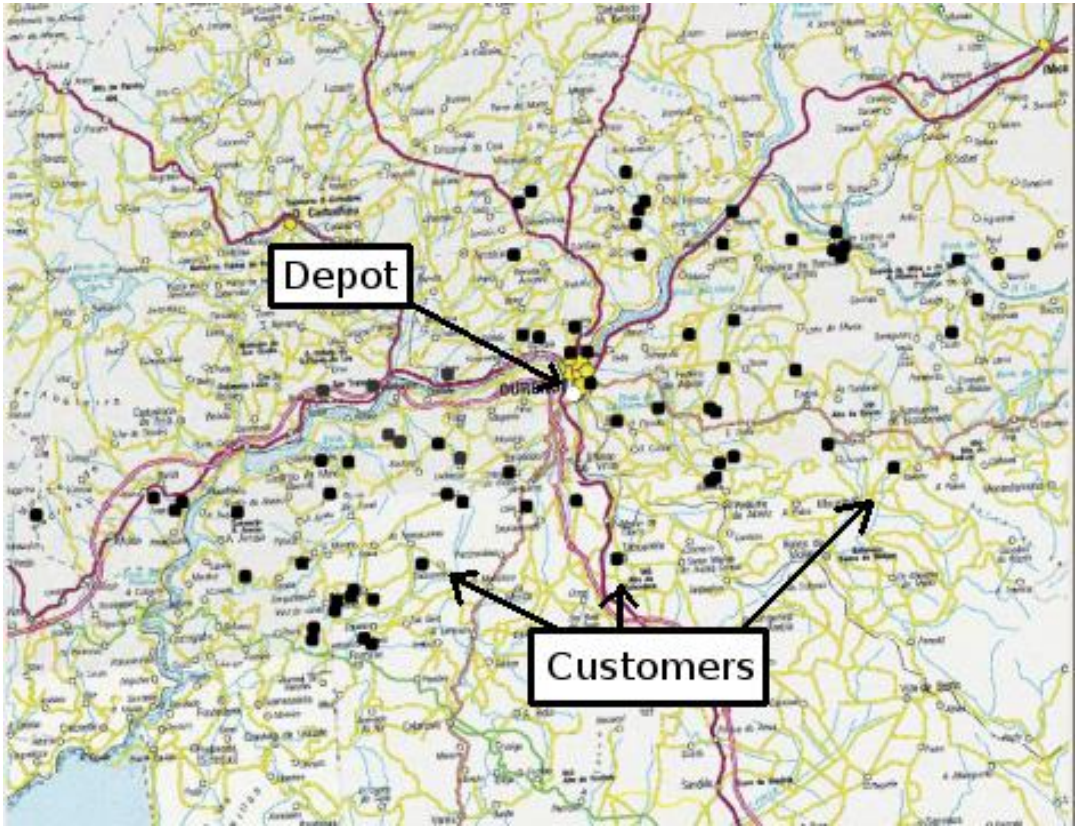
- How does the number of combinations to check grow?

The Vehicle Routing Problem

Task:

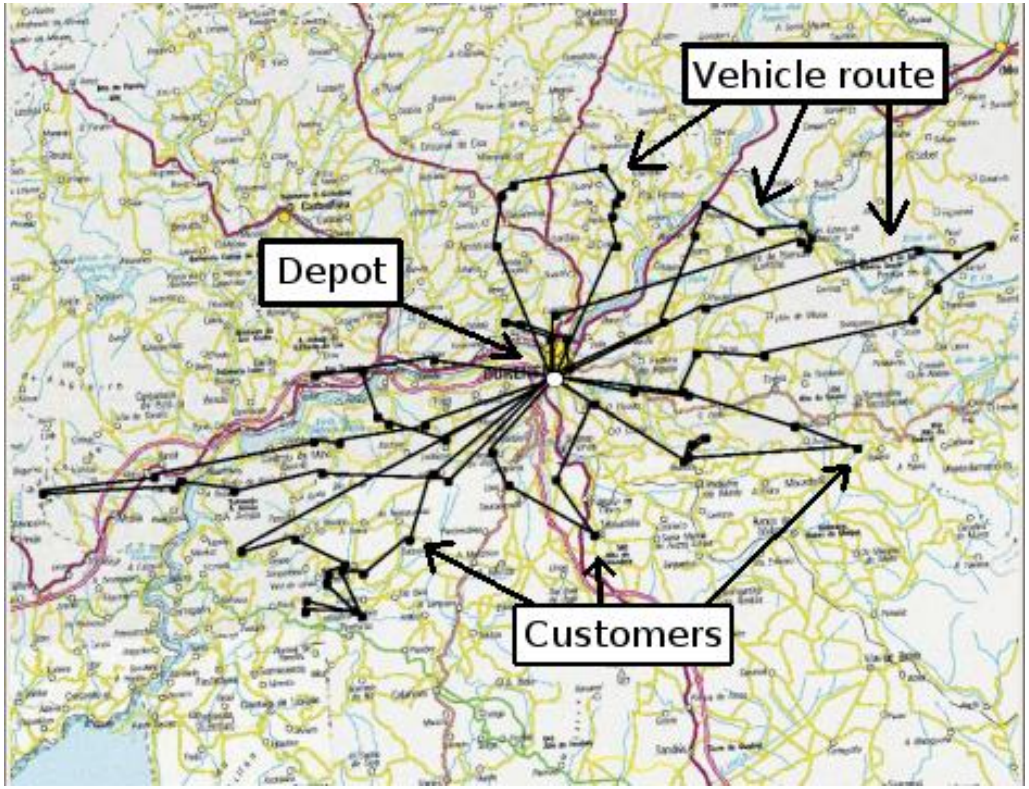
I have three vehicles.

Tell me how to route the vehicles so that I can deliver to all the customers, for the minimum fuel cost.



- What do I need to do to be sure I've found the best route?

The Vehicle Routing Problem

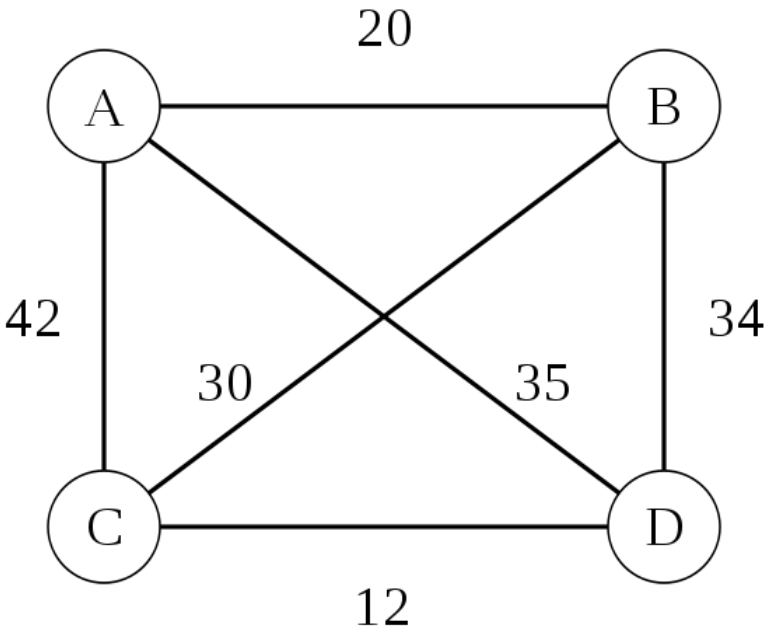


- To be **sure**, I need to evaluate every possible combination of routes my vehicles could take.

- Key feature:

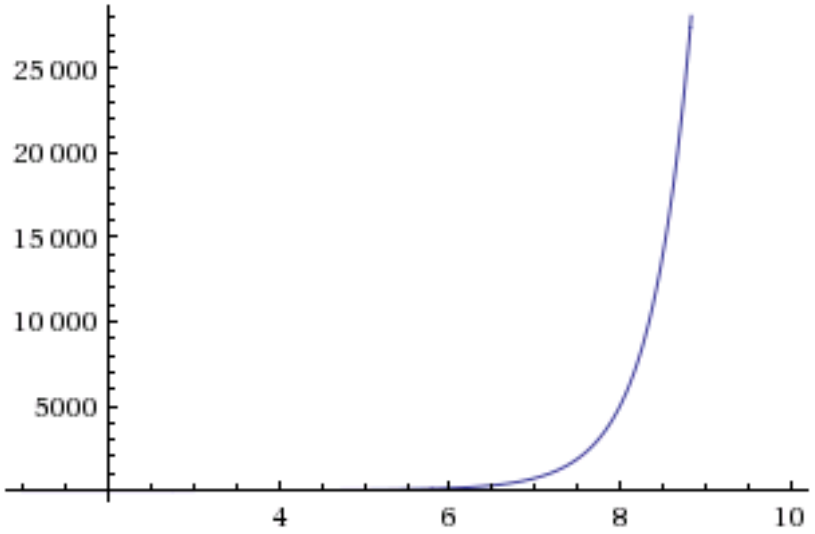
easy to evaluate how good a route is, but hard to **find** the best.

Let's abstract: The Travelling Salesperson Problem



- Task: Find the lowest cost route that visits all cities exactly once, and returns to the start point.
- 4 cities. How many possible routes if I start at A?
- ABCDA, ABDCA, ACBDA, ACDBA, ADBCA, ADCBA = 6
- In general, for a complete graph there are *(n-1)!* possible routes.

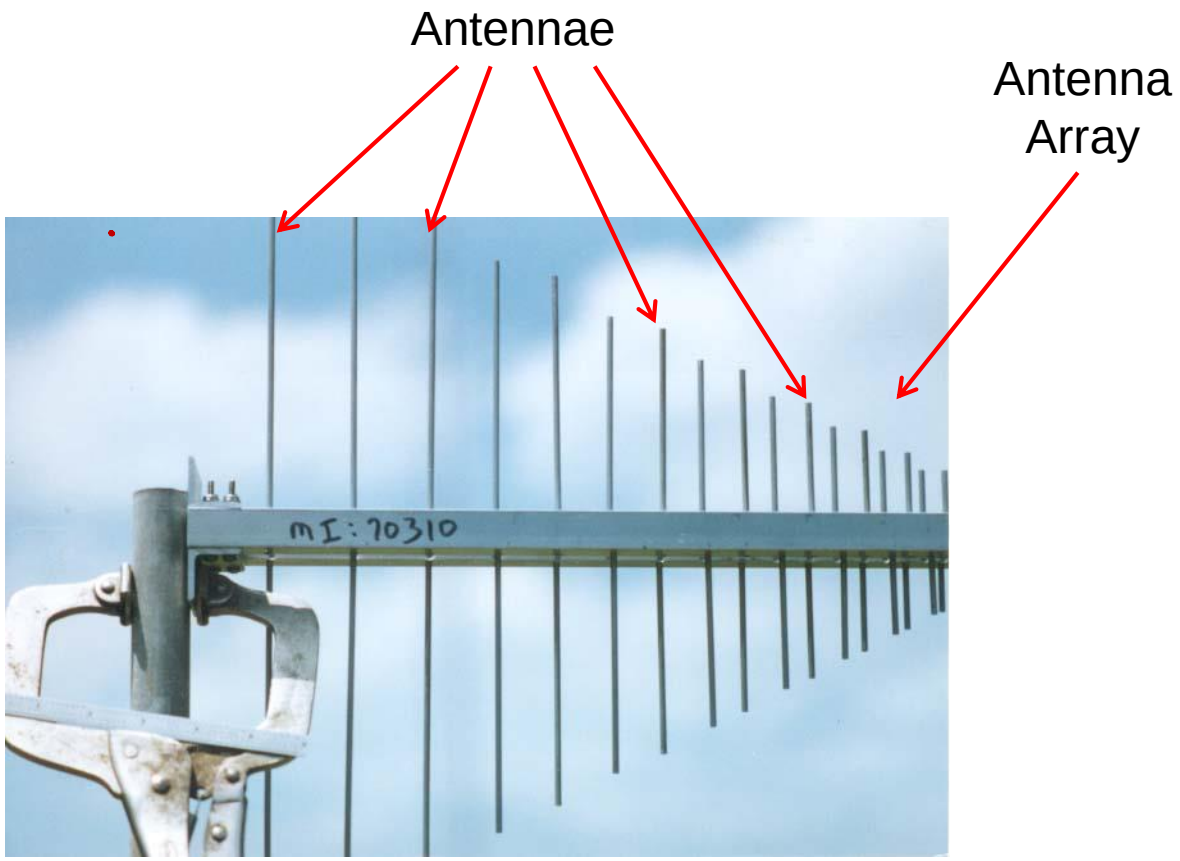
The Travelling Salesperson Problem: Scalability



Cities	Routes
5	24
10	362,880
15	~87 billion
20	~121 million billion

- Even for 20 cities, it is infeasible to check all the routes.
- E.g. the UK rail network has ~2,533 train stations. If I wanted to find the lowest cost route between them all, the number of routes I would (theoretically) need to check has 7524 digits in it!
- At a rate of 1 digit per centimetre, the number would be about the length of a New York City block.

Antenna Array Design



Antenna Array Design Problem

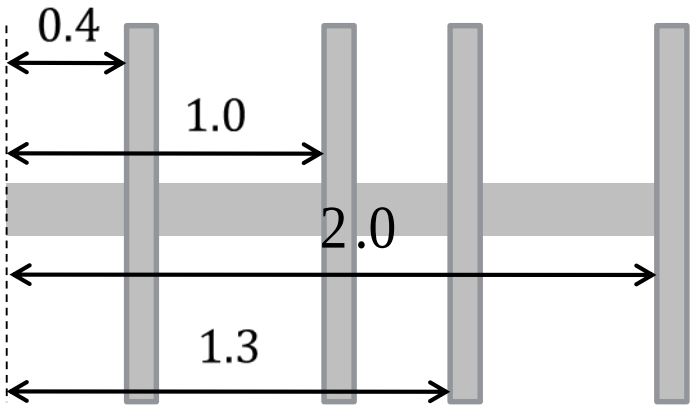


- **Have:** Fixed number of antennae which we can position on the antenna array (subject to **constraints**)
- **Aim**: Minimize peak Side Lobe Level (SLL) by choosing the position of each antenna.

Representing Antenna Arrays

- Natural way to represent the problem: distances from LHS of array.
- If we have 4 antennae to position on the array, one design could be:

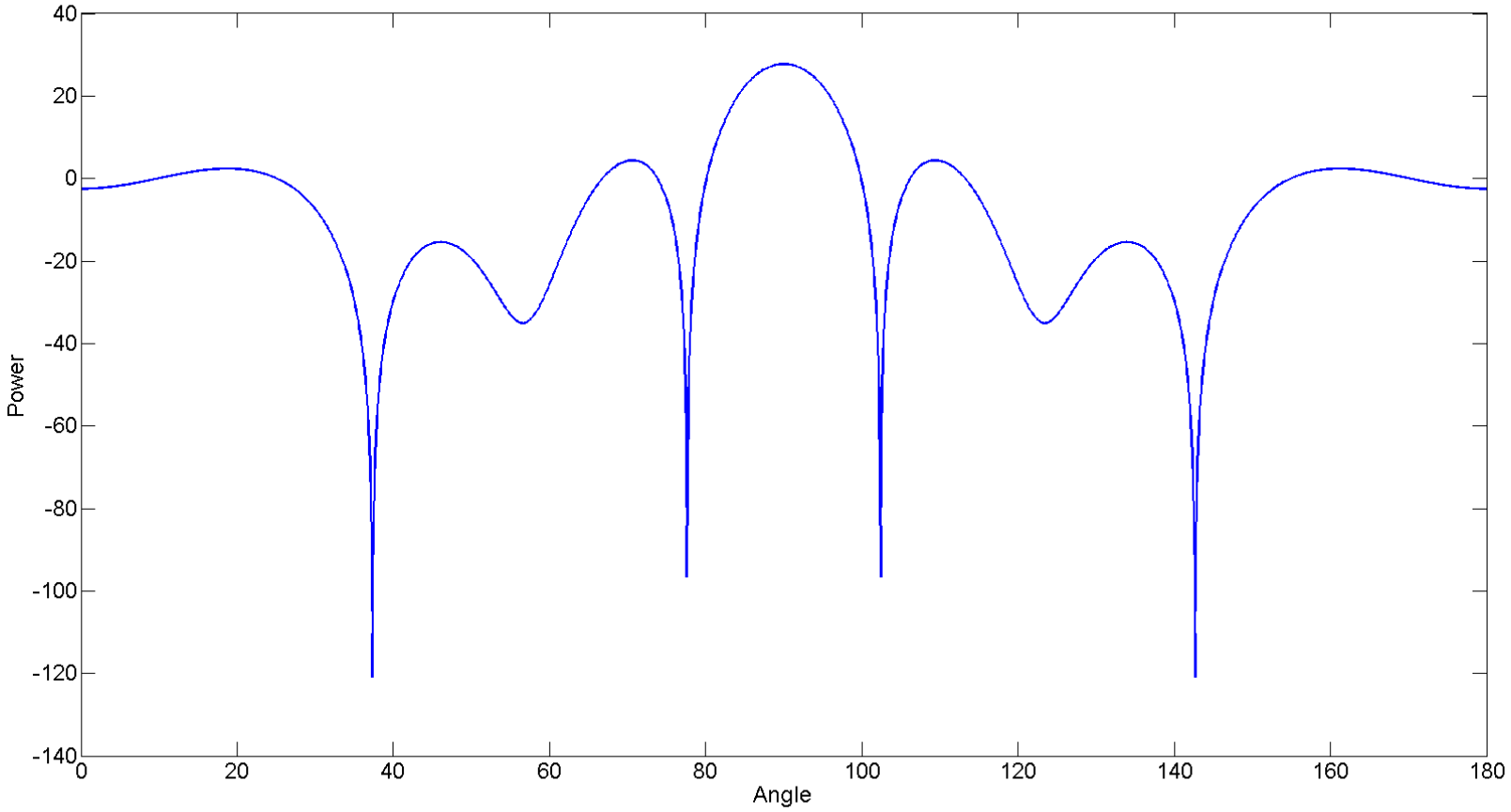
  [0.4, 1.0, 2.0, 1.3]



- Maximum value is called the **aperture size** – in this example 2.0

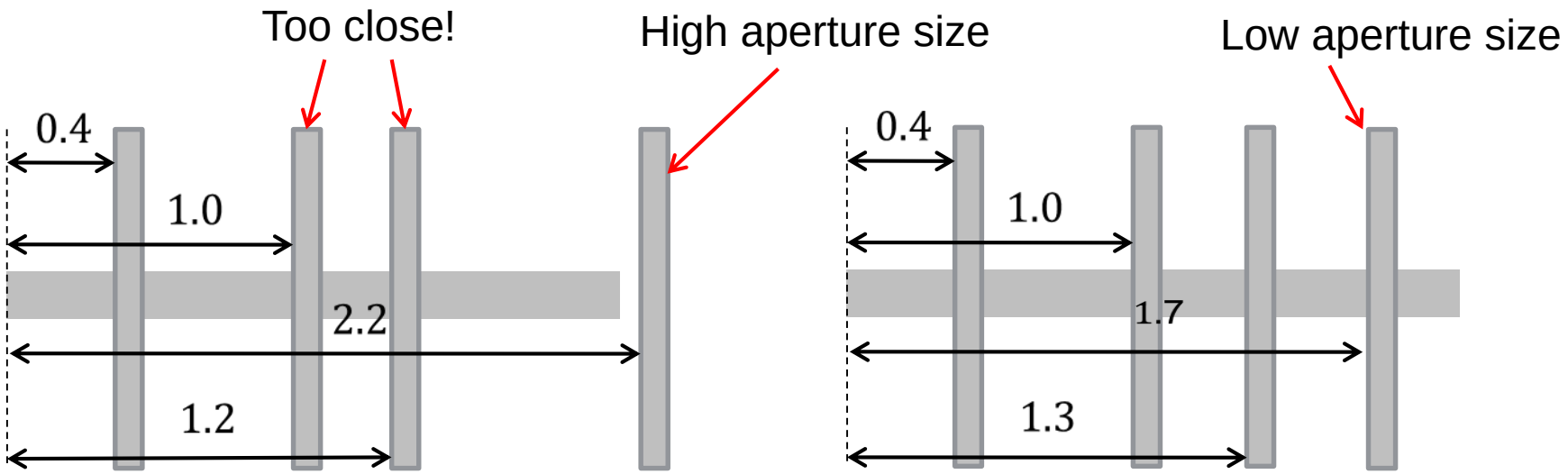
Effect of Antennae Placement

- The example design on the previous slide would have the following power curve:

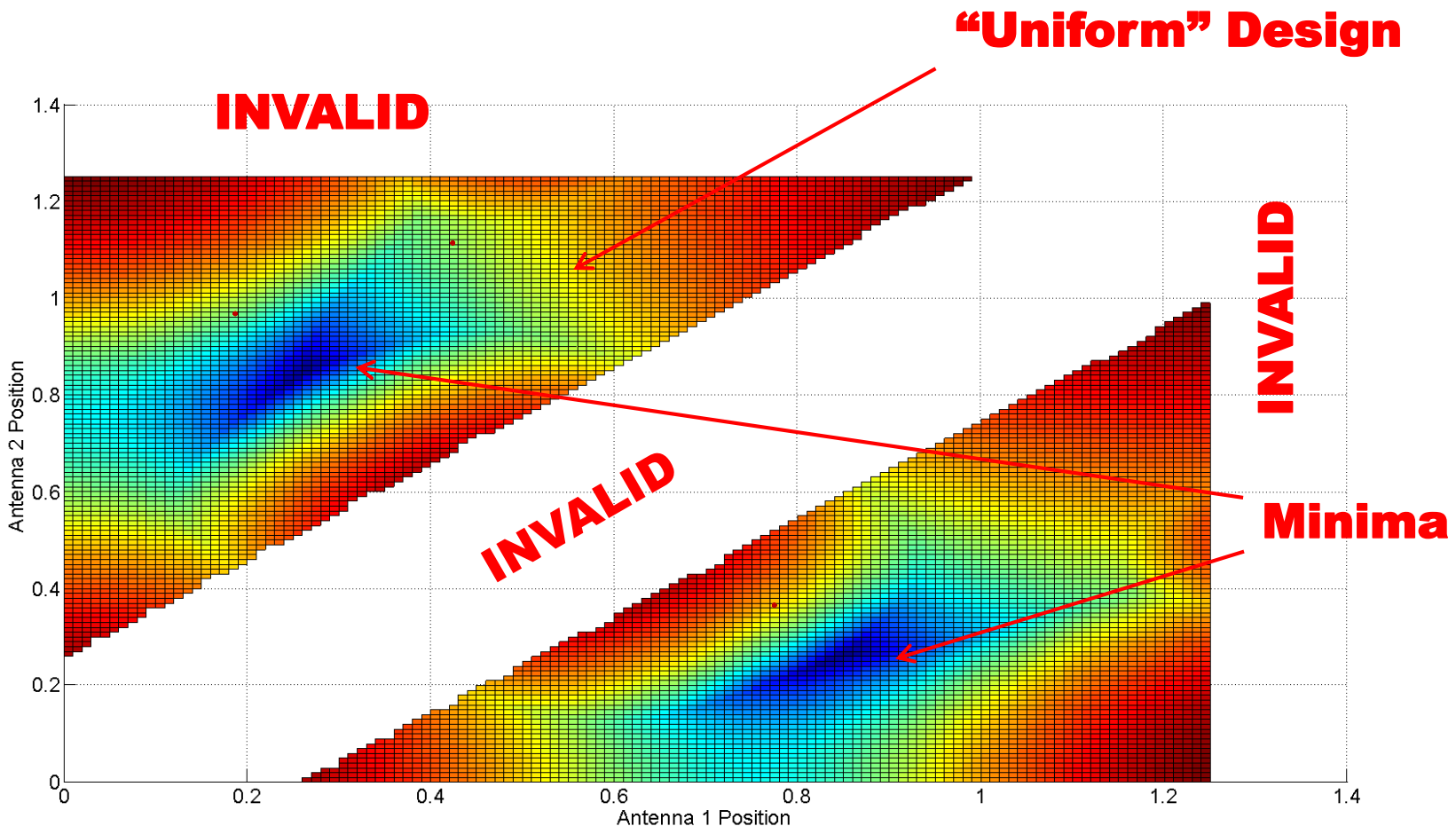


Constraints

- Not all representable antenna arrays are valid.
  - We want a fixed aperture size equal to **half the number of antennae**.
  - Antennae cannot be placed too closely to each other (**must be at least 0.25 apart**).



Visualising the Problem: Characteristics



So what makes a problem hard?

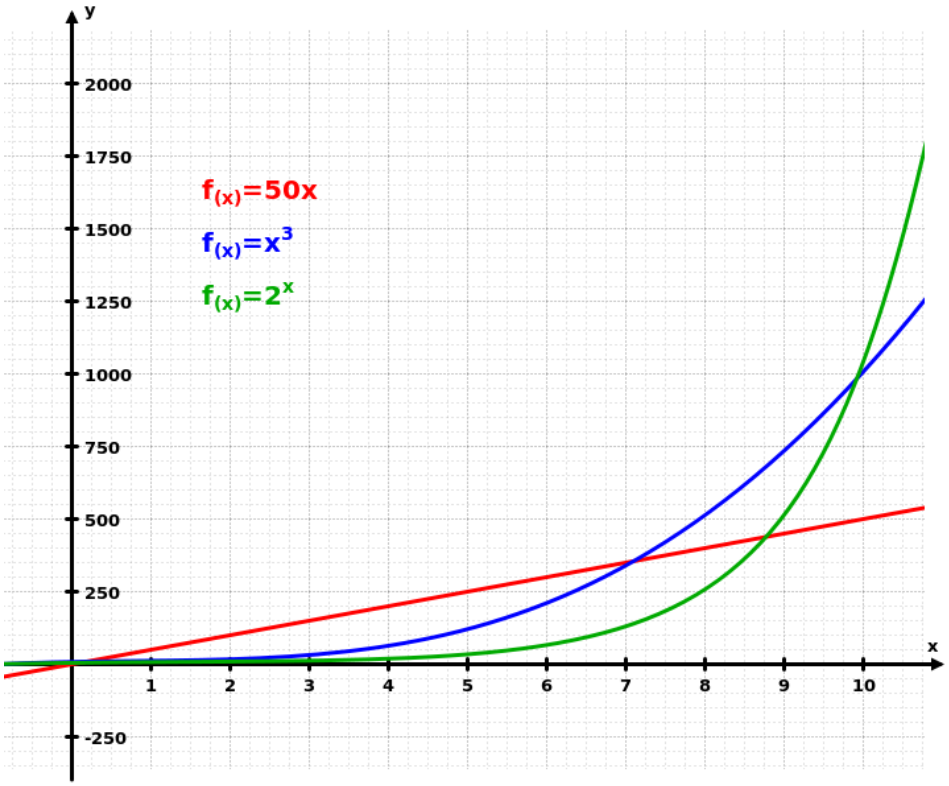


Finding a needle in a haystack is **not that hard**.

It scales **linearly** with the size of the haystack (n).

- The Travelling Salesman Problem is **a combinatorial problem** that scales according to a factorial function.
- The Antenna Array Design Problem is **continuous problem** that has a complex, unknown landscape.
- These are worlds apart from haystacks in terms of problem “hardness”.

Problem Scalability Rates



- Problems that scale linearly (red) and polynomially (blue) are “tractable”. We can solve them fully in reasonable amounts of time.
- Exponential (green) or factorial (not shown, even worse!) are **intractable**. While we can solve them in theory, the time and/or space requirements to check every solution are **utterly infeasible**.


ALL IS LOST

Not quite...



...to find out what we can do about it.

Summary of this lecture

- By the end of this lecture you should:
  - Understand what makes a “hard” problem, and be able to discuss the relative hardness of problems.
  - Know about the two main problems that will be used in this module:
    - The Travelling Salesman Problem, and
    - The Antenna Array Design Problem.
  - Appreciate the scale of each of these two problems, from the point of view of implementing solutions.